

Master Blueprint: FDE Legacy Enterprise Sandbox

1. Pre-Video Setup

These steps ensure the video focuses on engineering logic rather than waiting for data to load.

- The Infrastructure:** Spin up the Microsoft SQL Server (MSSQL 2022) Docker container on port `1433`.
- The Messy Schema:** Create `dbo_ops.TBL_SC_FLEET_HIST_RAW` using cryptic, legacy column names (`IOT_TEMP_VAL_C` , `CGO_COND_CD`).
- Data Ingestion:** Load the 15.4MB Southern California logistics CSV into the MSSQL table.
- Policy Document:** Pre-write `Cold_Chain_Incident_SOP_v2.md` with strict temperature and routing compliance rules.

2. In-Video Execution Roadmap

[Phase 1: Cold Open & The Messy Reality] → [Phase 2: FDE Security & Access] → [Phase 3: Architecture (TDD)]
|
[Phase 6: Frontend & Deployment] ← [Phase 5: LangGraph Orchestration] ← [Phase 4: Tool Construction]

Phase 1: The Cold Open (The Problem)

- The Scenario:** Introduce the business problem—cargo is spoiling in Southern California, and dispatchers are flying blind.
- The Reveal:** Open a SQL client (like DBeaver or Azure Data Studio). Run a `SELECT *` on the legacy MSSQL database.
- The FDE Callout:** Show the audience the cryptic columns (`IOT_TEMP_VAL_C` , `RISK_CLS_TXT`). Explain that standard RAG fails here, and giving an LLM direct access to an undocumented, live schema is a hallucination disaster waiting to happen.

Phase 2: FDE Security & Access (Live Execution)

- The Discovery Context:** Briefly state the constraint: "Because this is for internal analysts, we will use Text-to-SQL. But we cannot let the AI touch the root database."
- Security Implementation (Live Code):**
 - Write the SQL command to create a strictly scoped, read-only user (`USR_FDE_RO`).
- The Semantic View:** Create a SQL `VIEW` that translates the messy legacy columns into clean, LLM-friendly names (e.g., mapping `IOT_TEMP_VAL_C` to `Current_Temperature_Celsius`). Grant the read-only user access *only* to this view.

Phase 3: Architectural Design (TDD Overview)

- The Dual-Engine Approach:** Briefly show a single architecture slide. Explain how the agent will use SQL for the live telemetry, Vector Search for the SOP rules, and an external API for the weather.

Phase 4: Constructing the Agent Tools

- **Tool 1:** `query_telemetry_db(sql_query: str)**`
- Connect to MSSQL using `pyodbc` or `SQLAlchemy` under the `USR_FDE_RO` credentials.

- Execute the LLM-generated SQL against the clean semantic view.
- ****Tool 2:** `fetch_corridor_conditions(lat, lon)**`
- Query a mock/live weather API for traffic and storm data at the vehicle's coordinates.
- ****Tool 3:** `search_compliance_sop(query: str)**`
- Execute a vector search against the local `Cold_Chain_Incident_SOP_v2.md`.

Phase 5: LangGraph Agentic Orchestration

- **The Brain:** Wire the ReAct agent. Show how it reasons: *"I need to check the temperature via SQL -> It is high -> I need to check the SOP via RAG for the mitigation steps."*

Phase 6: Streamlit UI & Deployment

- **The Dashboard:** Launch the Streamlit app showing a chat interface on the left and a live SoCal PyDeck map on the right.
 - **The Climax:** Type the prompt: *"Identify refrigerated shipments on the Long Beach route with temperature anomalies and tell me the SOP protocol."*
 - **The Result:** Watch the agent query the MSSQL view, hit the weather API, read the SOP, output the mitigation plan, and plot the exact truck on the map.
-